02

Pandas

Your best tool for working with tables and data

AI/ML Engineer Learning Series · Deep Dive Edition

Topic	Pandas
Full Name	Python Data Analysis Library
Built On	NumPy (the one we learned last time)
Best For	Tables, rows, columns — like Excel but in Python
Used In	Data cleaning, EDA, feature engineering, ML pipelines

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	**What is Pandas?**	The Excel-in-Python idea
2	**Series & DataFrame**	The two main building blocks
3	**Reading Files**	Load CSV, Excel, JSON, SQL
4	**First Look**	head, info, describe, missing values
5	**Selecting Data**	loc, iloc, filtering with conditions
6	**Cleaning Data**	Missing values, duplicates, types
7	**Adding Columns**	Feature engineering, apply()
8	**Sorting & Ranking**	Order your rows
9	**GroupBy**	Group rows and do math on each group
10	**Merging Tables**	Join two tables together
11	**Text Columns**	Working with str functions
12	**Real ML Pipeline**	Full 7-step data prep example
13	**Common Mistakes**	What to avoid
★	**Cheatsheet**	Quick reference of every command

1. What is Pandas?

Imagine you have a big table — like in Excel. It has rows and columns. Each row is one person or one item. Each column is one type of info (like name, age, score). **Pandas** lets you do everything with that table inside Python — read it, clean it, find things, change things, and save it back.

In Machine Learning (■■■■■ ■■■■■■■ — ■■■■■■■■■■■ ■■■■■■), before you teach the computer anything, you first need to clean and prepare your data (■■■■ — ■■■■). Pandas is the tool everyone uses for that step.

> ■ *Think of Pandas like a really powerful Excel — but you control it with code, so you can do the same thing on 10 million rows in seconds.*

2. The Two Main Building Blocks

Series — One Single Column

A **Series** is just one column of data. It has an index (■■■■ — ■■■■■■■ ■■■■■■■ ■■■■■) on the left and values (■■■) on the right. Think of it like a list — but smarter, because each item has a label.

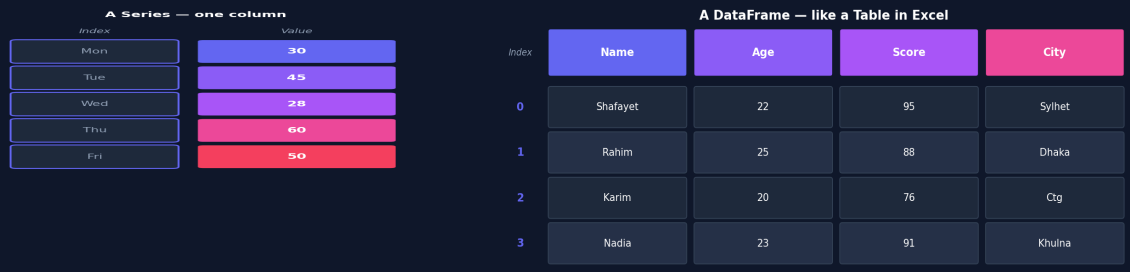

Fig 1 — Left: a Series (one column). Right: a DataFrame (many columns together).

```
import pandas as pd

# A Series — like a list with labels
scores = pd.Series([95, 88, 76, 91],
                   index=['Shafayet','Rahim','Karim','Nadia'])

print(scores['Rahim'])   # → 88
print(scores.mean())     # → 87.5
```

DataFrame — The Full Table

A **DataFrame** (■■■■ ■■■■■ — ■■■■■■ ■■■■■) is the full table — many columns put together. Each column is actually a Series. This is what you will use 90% of the time.

```
data = {
    'Name':  ['Shafayet','Rahim','Karim','Nadia'],
    'Age':   [22, 25, 20, 23],
    'Score': [95, 88, 76, 91],
    'City':  ['Sylhet','Dhaka','Ctg','Khulna']
}

df = pd.DataFrame(data)
print(df)
#      Name  Age  Score    City
# 0  Shafayet  22     95  Sylhet
# 1     Rahim  25     88   Dhaka
# ...
```

3. Reading Data From Files

Most of the time your data is not typed by hand. It is in a file — like a CSV file (a simple text file where values are separated by commas). Pandas can read many kinds of files with just one line.

File Type	Code	When to use
CSV (text table)	pd.read_csv('file.csv')	Most common — Kaggle datasets
Excel (.xlsx)	pd.read_excel('file.xlsx')	Business data, reports
JSON	pd.read_json('file.json')	Data from the web/APIs
SQL Database	pd.read_sql(query, connection)	Data from a database
Parquet	pd.read_parquet('file.parquet')	Big data, fast reading

```
# Read a CSV file
df = pd.read_csv('students.csv')

# Useful options
df = pd.read_csv('students.csv',
                 sep=';',          # if values separated by ; not ,
                 header=0,         # first row is column names
                 index_col='ID',   # use ID column as the index
                 nrows=1000)       # only read first 1000 rows

# Save back
df.to_csv('output.csv', index=False)  # index=False skips row numbers
```

4. First Look at Your Data

When you get a new dataset (■■■■■■■■ — ■■■■■■■ ■■■■■■■), the very first thing you do is look at it. These simple commands tell you what's inside.

Command	What it tells you
df.head(5)	Shows the first 5 rows — quick peek
df.tail(5)	Shows the last 5 rows
df.shape	How many rows and columns — e.g. (1000, 8)
df.columns	Names of all columns
df.dtypes	What type each column is (number, text, date...)
df.info()	Shape + dtypes + how many non-empty values
df.describe()	Min, max, mean, std for every number column
df.isnull().sum()	How many missing (■■■■) values per column
df.value_counts()	How many times each value appears

df.nunique()	How many unique (■■■■■) values per column

```
df = pd.read_csv('titanic.csv')

df.shape          # (891, 12)  → 891 rows, 12 columns
df.head(3)        # see first 3 rows
df.info()         # see column types and missing values
df.describe()     # see stats for number columns
df['Age'].isnull().sum()  # how many ages are missing → 177
```

5. Selecting Rows and Columns

Selecting (■■■■ ■■■■■■) means picking the part of the table you want. You can pick one column, many columns, one row, or rows that match a condition (■■■■).

Picking Columns

```
# One column → gives a Series
df['Name']

# Many columns → gives a DataFrame
df[['Name', 'Score', 'City']]
```

Picking Rows — loc and iloc

loc uses the label (■■■/■■■■■) to find rows. **iloc** uses the position number (■■■■■■■ ■■■■■ — 0, 1, 2...) to find rows. Think: loc = label, iloc = integer position.

```
# loc – use the label
df.loc[0]             # row with index 0
df.loc[0:2]           # rows 0, 1, 2 (inclusive! – □□□□□□ □□□)
df.loc[0, 'Name']     # row 0, column 'Name'
df.loc[:, 'Age':'Score']  # all rows, cols Age to Score

# iloc – use the position number
df.iloc[0]            # first row
df.iloc[-1]           # last row
df.iloc[0:3, 1:3]     # rows 0-2, cols 1-2 (not inclusive at end)
```

Filtering Rows with Conditions (■■■■ ■■■■■ ■■■■■)

This is one of the most useful things in Pandas. You ask: 'give me only the rows where Score is more than 80.' Pandas gives you exactly those rows.

```
# Only students with score above 80
df[df['Score'] > 80]

# Students from Sylhet AND score above 90
df[(df['City'] == 'Sylhet') & (df['Score'] > 90)]

# Students from Sylhet OR Dhaka
df[df['City'].isin(['Sylhet', 'Dhaka'])]

# Students whose name contains 'Rahim'
df[df['Name'].str.contains('Rahim')]
```

6. Cleaning Data (■■■■ ■■■■■■■■■ ■■■■)

Real world data is messy (■■■■■■■■). Some values are missing. Some are wrong. Some columns have the wrong type. Cleaning (■■■■■■■■■ ■■■■) means fixing all of this before you use the data.

Missing Values (■■■■ ■■■■)

```
# Find missing values
df.isnull().sum()              # count per column
df.isnull().sum() / len(df)   # percentage missing

# Option 1: Drop rows where Age is missing
df.dropna(subset=['Age'], inplace=True)

# Option 2: Fill missing Age with the average age
df['Age'].fillna(df['Age'].mean(), inplace=True)

# Option 3: Fill with the most common value
df['City'].fillna(df['City'].mode()[0], inplace=True)

# Option 4: Forward fill — use the value above
df['Score'].fillna(method='ffill', inplace=True)
```

Duplicates (■■■■ ■■■■■)

```
df.duplicated().sum()          # how many duplicate rows?
df.drop_duplicates(inplace=True)  # remove them
```

Wrong Data Types (■■■■ ■■■■)

```
# Age is stored as text — fix it
df['Age'] = df['Age'].astype(int)

# Score stored as text with % sign — remove % then convert
df['Score'] = df['Score'].str.replace('%','').astype(float)

# Convert a column to category type (saves memory)
df['City'] = df['City'].astype('category')
```

Renaming and Dropping Columns

```
# Rename columns
df.rename(columns={'Nm': 'Name', 'Sc': 'Score'}, inplace=True)

# Drop columns you don't need
df.drop(columns=['Useless_Col', 'Another'], inplace=True)

# Drop rows by index
df.drop(index=[0, 5, 10], inplace=True)
```

7. Adding and Changing Columns

You can make new columns from old ones. This is called **feature engineering** (■■■■■ ■■■■■■■■■■■■■■ — ■■■■■ ■■■■■ ■■■■■ ■■■■) in Machine Learning. You take what you have and make something new and useful.

```
# Add a new column
df['Passed'] = df['Score'] >= 50      # True/False

# Make a grade column from score
def get_grade(score):
    if score >= 90: return 'A'
    elif score >= 80: return 'B'
    elif score >= 70: return 'C'
    else: return 'F'

df['Grade'] = df['Score'].apply(get_grade)

# Faster way with np.where
import numpy as np
df['Senior'] = np.where(df['Age'] >= 25, 'Yes', 'No')

# Combine two columns
df['Full_Info'] = df['Name'] + ' from ' + df['City']

# Math on columns
df['Score_out_of_10'] = df['Score'] / 10
```

apply() — Do Anything to a Column

apply() lets you run any function on every value in a column. It is very flexible (■■■■■■■ — ■■■■■■■ ■■■■ ■■■■ ■■■■■).

```
# Apply a simple function
df['Name_Upper'] = df['Name'].apply(str.upper)

# Apply a lambda (□□□□□□□□□□ – □□□ □□-□□□□ □□□□□□)
df['Score_doubled'] = df['Score'].apply(lambda x: x * 2)

# Apply on whole rows (axis=1)
df['Summary'] = df.apply(
    lambda row: f"{row['Name']} scored {row['Score']}",
    axis=1
)
```

8. Sorting and Ranking (■■■■■■)

```
# Sort by Score — highest first
df.sort_values('Score', ascending=False)

# Sort by City first, then Score
df.sort_values(['City', 'Score'], ascending=[True, False])

# Add a rank column (1 = highest score)
df['Rank'] = df['Score'].rank(ascending=False).astype(int)

# Reset index after sorting
df = df.sort_values('Score', ascending=False).reset_index(drop=True)
```

9. GroupBy — Grouping Data (■■■ ■■■ ■■■)

GroupBy means: put rows into groups based on one column, then do some math on each group. Example: 'Give me the average score for each city.' Pandas splits the table by city, calculates the average, and gives you the result.

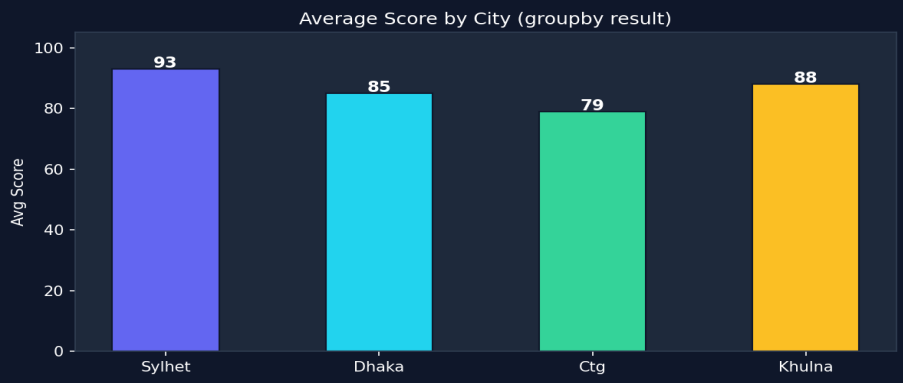

Fig 2 — Average score per city, calculated with groupby.

```
# Average score per city
df.groupby('City')['Score'].mean()

# Multiple stats at once
df.groupby('City')['Score'].agg(['mean', 'min', 'max', 'count'])

# Group by two columns
df.groupby(['City', 'Grade'])['Score'].mean()

# Apply different functions to different columns
df.groupby('City').agg({
    'Score': 'mean',
    'Age':   'max',
    'Name':  'count'
})
```

■ *GroupBy is one of the most used things in EDA (Exploratory Data Analysis — ■■■■ ■■■■■■■ ■■■■■■■■■). Almost every time you want to understand your data, you use groupby.*

10. Merging Tables (■■■■ ■■■■■ ■■■■■ ■■■■■■)

Sometimes your data is in two different tables and you need to join (■■■■■■) them. Just like in a database. Pandas makes this easy with **merge()**.

```
students = pd.DataFrame({
    'ID':   [1, 2, 3],
    'Name': ['Shafayet','Rahim','Karim']
})

scores = pd.DataFrame({
    'ID':    [1, 2, 3],
    'Score': [95, 88, 76]
})

# Join them on the ID column
result = pd.merge(students, scores, on='ID')
#    ID      Name  Score
# 0   1  Shafayet     95
# 1   2     Rahim     88
# 2   3     Karim     76
```

Merge Type	What it does
inner (default)	Only keep rows that exist in BOTH tables
left	Keep all rows from left table, match what you can from right
right	Keep all rows from right table
outer	Keep ALL rows from both tables (fill missing with NaN)

```
# Left join — keep all students even if no score
pd.merge(students, scores, on='ID', how='left')

# Stack tables on top of each other (same columns)
pd.concat([df1, df2], ignore_index=True)
```

11. Working with Text Columns (str functions)

When a column has text (■■■■) in it, you use **.str.** to do things with it. These are the same operations as regular Python string methods — but they work on the whole column at once.

```
df['Name'].str.upper()          # SHAFAYET, RAHIM...
df['Name'].str.lower()          # shafayet, rahim...
df['Name'].str.len()            # number of characters
df['Name'].str.strip()          # remove spaces from edges
df['Name'].str.replace('a','@') # replace letter
df['Name'].str.startswith('S')  # True/False
df['Name'].str.contains('ahi')  # True/False
df['Name'].str.split(' ')       # split into list of words
df['City'].str[:3]              # first 3 characters
```

12. A Real ML Data Prep Pipeline

Here is a real example of how you use Pandas to prepare data before training a model. This is the kind of code you write every time you start a new ML project.

```
import pandas as pd
import numpy as np

# Step 1: Load
df = pd.read_csv('titanic.csv')

# Step 2: Quick look
print(df.shape)          # (891, 12)
print(df.isnull().sum()) # see missing values

# Step 3: Drop columns we don't need
df.drop(columns=['Name','Ticket','Cabin'], inplace=True)

# Step 4: Fill missing values
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

# Step 5: Convert text to numbers
# (models only understand numbers — not 'male'/'female')
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Step 6: One-hot encoding
# (□□□□□-□□ □□□□□□□ — □□□□□□ □/□ □□□□□ □□□□□□)
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)

# Step 7: Split into features and target
X = df.drop('Survived', axis=1)  # input columns
y = df['Survived']               # what we want to predict

print(X.shape)   # ready for ML model!
```

■ *This 7-step pattern is what you do at the start of almost every ML project. Load → Look → Drop → Fill → Encode → Split. Learn it by heart.*

13. Common Mistakes (■■■■■■ ■■■)

Mistake	What Happens	Fix
Forgetting inplace=True	Your change is not saved to df	Write inplace=True OR df = df.something()
SettingWithCopyWarning	You edit a copy, not the real df	Use df.loc[row, col] = value
Wrong column name	KeyError crash	Check df.columns first
Chained indexing df[..][..]	Might edit a copy silently	Always use .loc[row, col]

Forgetting reset_index	Index jumps 0,3,7... after filtering	df.reset_index(drop=True, inplace=True)
apply() is slow on big data	Takes a long time on millions of rows	Use vectorised operations where possible

Quick Reference Cheatsheet

Task	Code
Read CSV	pd.read_csv('file.csv')
See first rows	df.head()
Shape	df.shape
Column types	df.dtypes
Missing count	df.isnull().sum()
Select column	df['col'] or df[['a','b']]
Filter rows	df[df['col'] > 5]
Select by label	df.loc[row, 'col']
Select by position	df.iloc[0, 2]
Drop missing	df.dropna(subset=['col'])
Fill missing	df['col'].fillna(value)
Remove duplicates	df.drop_duplicates()
Change type	df['col'].astype(int)
Add new column	df['new'] = df['a'] + df['b']
Apply function	df['col'].apply(func)
Sort	df.sort_values('col', ascending=False)
Group and aggregate	df.groupby('col')['val'].mean()
Merge two tables	pd.merge(df1, df2, on='key')
Stack tables	pd.concat([df1, df2])
One-hot encode	pd.get_dummies(df, columns=['col'])
Map text to number	df['col'].map({'a':0,'b':1})
Value counts	df['col'].value_counts()
Save to CSV	df.to_csv('out.csv', index=False)

Next Topic: Matplotlib & Seaborn — drawing charts and graphs so you can *see* your data. After Pandas cleans the data, Matplotlib shows it visually.